Deep Dive D — Subtyping Validation Soundness: `Clauses::isSub` and Its Strata

Models revision `bafdcf7c` of `main` (2026-08-05).
Companion to *Refinement and Inference Growth in SKJS*.

Abstract

This deep dive addresses the largest unproved component of the main soundness paper: Obligation **L-SUB** (*Validation Soundness*), the claim that acceptance by the subtyping validator implies semantic inclusion under the current substitution state. Part I proves the architecture: the `isSub` dispatch, the two memoization caches (`subst.seenSub` and the version-keyed `tenv.seenSub`), the growth pre-passes, restoration after failed checks, and the coinductive treatment of recursive types. A stratified-semantics theorem reduces the recursive case to soundness of each individual representation layer. Part II establishes the layers supported by the main paper's membership semantics: primitives, STypes, DNF clause covering, object-branch matching, and the getter, optional-field, index, and access rules of TFields. Setter rules remain conditional on **L-MUT**; function rules are checked against the behavioral signature semantics recorded by **E-SIG**.

The analysis finds four places where acceptance is not justified by those semantics. A callable can pass through `Function` and {} into a type marked non-callable (**F-CABSENT**); the array brand is never compared (**F-BRAND**); one path accepts after dropping a type-parameter constraint; and subtyping between two implicit shape receivers, or two implicit class receivers, does not compare their receiver types (**F-THIS-IMPLICIT**). The registry separates these findings and the remaining conditional cases from the parts of L-SUB established here.

Contents

1 Purpose: the Row Addressed **2**

2 Minimal Recap **3**

I Architecture and Caches (Proved) **4**

3 The Dispatcher and Its Acceptance Points **4**
3.1 Restore-on-failure 6
3.2 The proved-pair memo: `subst.seenSub` 6
3.3 Growth pre-passes are guidance 7

4 The Cycle Cache: `tenv.seenSub` and Version Keying **7**
4.1 The write audit: every binding write bumps the version 7
4.2 Why the naive reading fails, and what replaces it 8
4.3 Path scoping 8

4.4 Record-point coherence . . . 8
4.5 The head-depth refusal . . . 9

5 Coinduction over the Value Semantics **9**
5.1 What the semantics of a recursive type even is . . . 9
5.2 One-step soundness, and the reduction . . . 10

II The Strata **11**

6 Primitives: Units and `TPrimValue::isSub` **11**

7 The Union Spine: `STypes::isSub` **11**

8 DNF Covering: `Clauses::isSub` **12**

9 Object Branches: `TObject::isSub` **12**

10 Fields: `TFields::isSub` **13**
10.1 Shared fields (Both) . . . 13
10.2 Sub-only fields (Left) and super-only fields (Right) . . . 14
10.3 Index pairs and the covering semantics . . . 15
10.4 Callability: the lattice check and a violation . . . 16
10.5 The array brand: an omission . . . 17

11 Functions: `TFun::isSub` and a Behavioral Signature Semantics **17**

12 Variance Plumbing: Why Each Side Expands the Way It Does **19**

13 Status and Recommended Ledger Edits **20**
13.1 Status by stratum . . . 20
13.2 Stratum outcomes and recommendations . . . 21
13.3 Failed-attempt log . . . 23

1 Purpose: the Row Addressed

The main paper's assumption ledger carries one row for the entire validator:

> **L-SUB** (Obligation "Validation Soundness"; the main paper's table records the split status this paper establishes): `Clauses::isSub` (with its STypes, object-branch, field, index, and function strata, including getter/setter variance and the `seenSub`/`tenv.seenSub` cycle caches keyed by substitution version) is sound in the sense of the main paper's Definition "Subtyping Check Architecture": if `Clauses::isSub` accepts under state ρ, then $[\![t_1]\!]_\rho \subseteq [\![t_2]\!]_\rho$ for the expanded views. *"This is the largest unproved component of the document."*

This paper is the discharge attempt. The outcome, in one paragraph: the *architecture* (caches, dispatch, coinduction) is proved outright, modulo two small named assumptions; the *covering skeletons* (STypes, Clauses, TObject) and the *read-direction* field rules are proved against the main paper's membership semantics; the *write-direction* rules are proved conditional on the mutation model (**L-MUT**); *function* subtyping is proved against a new behavioral signature semantics that

we propose as an extension of the main paper's object semantics (whose opaque signatures cannot even express the claim); and two arms of the field stratum — the lax ($\mathsf{CUnknown}, \mathsf{CAbsent}$) callability arm and the array *brand* — are found to be unsound relative to the main paper's stated semantics, each composing with a refinement rule whose proof in the main paper leans on exactly the semantic clause the subtyping arm ignores — the first witnessed end-to-end, the second by composite sketch (Findings 30 and 31). Section 13 gives the resulting ledger split.

Relation to the rest of the series. We build on three dives (exact rows in this paper's Imports table): Deep Dive B (`lattice-operations.tex`) for the union/intersection results — union proved exact, and the $\bot$-clause refuted in its live strata (incomparable signatures and beyond-precision num-spelling, **L-INTER-BOT**); Deep Dive C (`type-expansion.tex`) for **L-EXP** scoped to expansion modes `None`/`Covariant` (its theorem T-EXP), which explicitly delegates the *contravariant* expansion read to this paper (§12); and Deep Dive E (`subst-state-discipline.tex`) for the substitution state discipline: monotone reads and their transport theorem proved, with the named residue MR-1 (the freeze-window memo hazard, which lives in `subst.seenSub` and is *inherited* here, §3.2), and the refutation of the naive "version is a state hash" reading of the `tenv.seenSub` key, whose corrected argument we complete in §4. All code references are to revision `bafdcf7c`; the primary sources are `src/TTypeSubtyping.sk` (all 2203 lines audited), `src/TTypeSubst.sk`, and the `isSub` helpers in `src/TType.sk`.

2 Minimal Recap

Representation. From the main paper (§3): a TType is a pair (I, C) of an instance layer $I \in \mathsf{TInsts}$ (type variables `TVar`, unexpanded named instantiations `TSInst`, unexpanded intersections `TInter`, delayed array literals `TArrayLit`) and clauses $C \in \mathsf{Clauses}$, a DNF: a map from var-sets to clauses $\langle V \mid S\rangle$ with $V \subseteq_{\text{fin}} \mathsf{TParam}$ and $S \in \mathsf{STypes} \cup \{\top\}$. STypes is tag-indexed, at most one SType per tag; STypes are units, literal-or-full primitives, and $\mathsf{object}(B)$ with branches $b \in \mathsf{TFields}$ carrying named string/symbol fields, per-kind indexes, call/new *callability modes* — the four-point lattice $\mathsf{Callability} = \{\mathsf{CAbsent}, \mathsf{CUnknown}, \mathsf{CFunction}, \mathsf{CPresent}(\sigma)\}$, ordered $\mathsf{CPresent} \le \mathsf{CFunction} \le \mathsf{CUnknown}$ and $\mathsf{CAbsent} \le \mathsf{CUnknown}$ (the main paper's Definition "Callability"; `src/TType.sk:2217-2237`) — and the array brand $\mathsf{isArr} \in \{\mathsf{None}, \mathsf{Some}(\mathsf{true}), \mathsf{Some}(\mathsf{false})\}$. Two invariants are enforced at $\mathsf{TFields}$ construction (`src/TType.sk:2266-2281`): an array-branded branch lowers an unconstrained mode to $\mathsf{CAbsent}$, and a constructible branch has $\mathsf{call} \ge \mathsf{CFunction}$ (ECMA-262: every value implementing [[Construct]] implements [[Call]]). Semantics ($[\![\cdot]\!]_\rho$, indexed by the substitution state ρ):

$$[\![(I, C)]\!]_\rho = [\![I]\!]_\rho \cup [\![C]\!]_\rho, \qquad [\![C]\!]_\rho = \bigcup_{c \in C} [\![c]\!]_\rho, \qquad [\![\langle V \mid S\rangle]\!]_\rho = \bigcap_{v \in V} [\![v]\!]_\rho \cap [\![S]\!]_\rho,$$

with $[\![S]\!] = \bigcup_{s \in S} [\![s]\!]$ and object membership per the main paper's Definition "Object-Type Semantics": required/optional fields constrain values via get-types, indexes constrain the props they *cover* (string index covers both string- and numeric-kind string keys; number index covers numeric-kind keys; symbol index covers symbol keys), $\mathsf{call}(b) = \mathsf{CAbsent}$ entails $\neg\mathsf{callable}(o)$, $\mathsf{CFunction}$ and $\mathsf{CPresent}(\sigma)$ entail $\mathsf{callable}(o)$, $\mathsf{CUnknown}$ entails nothing — dually for new and $\mathsf{constructible}$ — and $\mathsf{isArr}(b) = \mathsf{Some}(x)$ entails $\mathsf{isArrayV}(o) = x$. Signatures σ are *opaque* to membership — until §11, where we refine this.

Check architecture. The main paper's Definition "Subtyping Check Architecture": `TType::isSub`(t_1, t_2) runs (1) growTVars$(t_1, t_2, \text{contra})$, (2) growTVars$(t_2, t_1, \text{cov})$, (3) expansion of both sides and validation by `Clauses::isSub`; on failure the substitution is restored. The Guidance-vs-Validation theorem there shows that *given* **L-SUB**, no imprecision of growth can cause unsound acceptance: growth selects the state, validation decides under it. This paper is about the "validation decides" half.

Target statement. Throughout, *acceptance* means the top-level `TType::isSub`(ρ, t_1, t_2) returning STOk, and the target is:

> **L-SUB (restated).** If `TType::isSub`(t_1, t_2) returns STOk, evolving the substitution $\rho \to \rho^*$, then $[\![t_1]\!]_{\rho^*} \subseteq [\![t_2]\!]_{\rho^*}$.

Two scoping remarks. (i) The statement quantifies over the state *produced* by the accepting run; persistence of the inclusion under *later* evolutions is Deep Dive E's monotone-reads theorem, not this paper. (ii) For types containing inference variables, $[\![t]\!]_\rho$ depends on the semantics assigned to a variable occurrence; we make the required directional reads explicit in §12 rather than burying them in the notation.

Result vocabulary. `STResult` is STOk or an STError tree. The error hierarchy (several hundred of the file's 2203 lines) exists to *render* failures; it never influences acceptance. We audit it only for state effects: `toError` renderers take the substitution and may expand types for printing, but every render site on the acceptance path occurs strictly after the verdict is fixed, and `originSuggestions` explicitly works on a `subst.clone()`. Rendering is therefore out of scope for L-SUB, with one caveat folded into the audit of Lemma 5.

impl: `TTypeSubtyping.sk / STError hierarchy, originSuggestions`

Part I
Architecture and Caches (Proved)

3 The Dispatcher and Its Acceptance Points

Definition 1 (Dispatch)**.** *`TType::isSub`(u_1, u_2) is a thin wrapper: (a) if $u_1 = u_2$, accept; (a′)* reflexive-`TSInst` fast path*: if u_1 and u_2 are clause-free single `TSInst`s of the same name value and equal arity with each argument pair equal — syntactically, or u_1's argument a variable that `peekRaw`-resolves to u_2's — accept; (b) if $(u_1, u_2) \in$ `subst.seenSub`, accept; (c) call `isSub_`, and on* STOk *insert (u_1, u_2) into `subst.seenSub`. `isSub_`(u_1, u_2) in order: (1) if u_2.isTop(), accept; (2) if* $(\text{version}, u_1, u_2) \in$ *`tenv.seenSub`, accept; otherwise — after the head-depth refusal of Definition 11 — record* $(\text{version}, u_1, u_2)$ *in the* local *`tenv` copy; (3) if u_1 is a single `TInter`(tys) with $u_2 \in tys$, accept; (4) if `ArrayLitTyping::subtyping` claims the pair, return its verdict; (5) save; run the two growth pre-passes; expand u_1 covariantly and u_2 contravariantly; validate with `Clauses::isSub`; restore on failure.*

impl: `TTypeSubtyping.sk / TType::isSub, TType::isSub_`

There are thus exactly eight ways to accept: (a), (a′), (b), (1)–(4) accepting, and (5) validating. We prove (a), (1), (3) here; (a′) is examined here and carries a *violated* transport premise, witnessed

(Lemma 4, registry row **L-SUB-TSINST**); (b) is §3.2; (2) in §4 and §5; (5) is Part II. (4) is *delegated*:

Obligation 2 (O-ARRAYLIT: Delayed Array-Literal Subtyping)**.** *`ArrayLitTyping::subtyping` decides pairs involving delayed array-literal forms (committing their tuple-vs-array shape) before the general machinery runs. Its acceptance must imply inclusion, and its failure paths must restore (the code saves/restores around its probes). Deep Dive C's assumption* ***E-ARR*** *covers the* expansion *of committed literals; the* decision procedure *itself remains open. Status:* OPEN.
impl: `TArrayLitTyping.sk / ArrayLitTyping::subtyping`

Assumption 3 (A-HASH: Structural Identity via Hashing)**.** *`TType::compare` and `TType` equality are implemented as comparison of the 64-bit `XXHash` computed by the smart constructors (`TType.sk / compare`). Every set and map keyed by* TType *in this paper — the $u_1 = u_2$ fast path, both `seenSub` caches, `seenExpansion` — therefore identifies types up to hash collision. We assume no two distinct constructed types collide. This is the standard hash-consing wager; it is stated once here and not repeated at each use. The reflexive-`TSInst` arm (a') adds a second identity notion — equality of name* value *plus `peekRaw`-resolution of argument variables — finer than raw hash equality, analyzed directly in Lemma 4 rather than resting on this wager. (A collision would be a* soundness *event: the fast path would equate distinct types.) Status:* ASSUMED.

Lemma 4 (Fast Path, Top, and the TInter Shortcut)**.** *Acceptances (a), (1), (3) are sound. Acceptance (a') satisfies the verdict-state inclusion when the resolved argument variables are unconstrained; its transport premise is* violated*, with an end-to-end witness (registry row* ***L-SUB-TSINST****).*

Proof. (a) $u_1 = u_2$ (modulo A-HASH) gives $[\![u_1]\!]_\rho = [\![u_2]\!]_\rho$ for any ρ: the denotation is a function of the term and the state (of the term *including its position's file* — name resolution keys on the referencing file — a scope shared by every position-blind identity in this paper and folded into A-HASH's wager), and no state evolution occurs on this path. (a') u_1, u_2 are clause-free with single instances $\mathsf{TSInst}(n_1, \bar{t}_1)$, $\mathsf{TSInst}(n_2, \bar{t}_2)$; the arm accepts only when $n_1.\mathsf{value} = n_2.\mathsf{value}$, $|\bar{t}_1| = |\bar{t}_2|$, and for each i either $t_{1,i} = t_{2,i}$ or $t_{1,i}$ is a lone variable with $\mathtt{peekRaw}(t_{1,i}) = t_{2,i}$. Three facts bound what this buys. *(Verdict state, unconstrained.)* `peekRaw` returns the variable's current binding without the constraint intersection of `Subst::get`; for an occurrence with $tv.\mathsf{constraints} = \emptyset$ it denotes exactly $[\![t_{2,i}]\!]_\rho$, so pointwise-equal arguments give $[\![u_1]\!]_\rho = [\![u_2]\!]_\rho$ at the verdict state. *(Constrained occurrences.)* With $tv.\mathsf{constraints} = C \neq \emptyset$ the occurrence denotes $[\![t_{2,i}]\!]_\rho \cap \bigcap [\![C]\!]$, not $[\![t_{2,i}]\!]_\rho$: the equality — which is what lets the arm ignore the variance of the argument position — fails as stated, leaving only an inclusion that invariant and contravariant argument positions do not license. *(Transport.)* The equality is read off an *unsealed* SVOpen candidate. The slow path's covariant grow seals what it reads (`isSub_` step (2); `Subst::get` covariant, SVOpen → SVClosed, `TTypeSubst.sk:218-227`); the arm performs the exact-value read *without* the seal — precisely the read discipline the main paper's monotone-reads obligation forbids for open candidates — so a later contravariant grow widens the binding and the verdict-state equality is not transported. This is not hypothetical: an inferred `Box<number>` aliased through the arm, then widened by `b.set("s")`, yields an accepted program in which a value typed `number` holds the string `"s"` at runtime (issue draft `issues/reflexive-tsinst-fastpath-unsealed-read.md`, runnable repro). The arm returns STOk without expanding or growing (without inference guidance, state writes, or memoization) — which is why nothing seals the peeked variable. Fix directions: seal the peeked variables (a covariant `get`) before accepting; restrict the arm to SVClosed bindings; or retire the arm. (1) $u_2.\mathsf{isTop}()$ holds iff the clause map is the singleton top clause $\langle \emptyset \mid \top \rangle$ (`Clauses::isTop`),

whose denotation is Val; note `TType::create` deletes the instance layer when the clauses are top, so $[\![u_2]\!]_\rho = \mathsf{Val} \supseteq$ anything, and again no evolution occurs. (3) requires $[\![\mathsf{TInter}(tys)]\!]_\rho \subseteq [\![u_2]\!]_\rho$ when $u_2 \in tys$ and u_1's clause part is empty (`getSingleInter` demands a lone instance). An unexpanded intersection denotes at most the intersection of its conjuncts — this is the instance-layer counterpart of Deep Dive C's expansion semantics for `TInter` (its `TInter::expand` computes the conjunction), and we record it as the interface fact *A-INTERSEM*: $[\![\mathsf{TInter}(tys)]\!]_\rho \subseteq \bigcap_{t \in tys} [\![t]\!]_\rho$. Given A-INTERSEM the shortcut is immediate, and it performs no state write. □

3.1 Restore-on-failure

Lemma 5 (Error Paths Restore)**.** *Every function of the* ***isSub*** *family returns* STError *only with the substitution equal to its entry state. Audit, function by function:* ***TType::isSub_*** *saves before growth and restores in its single error arm;* ***Clauses::isSub***, ***STypes::isSub***, ***TObject::isSub***, ***TFields::isSub***, *and* ***TFun::isSub*** *each save at entry and restore before returning any error (****TObject::isSub*** *additionally save/restores around each failed branch pairing);* ***TPrimValue::isSub*** *takes no substitution; the probes inside* ***growTVars*** *(****isCovered****, the constraint-admissibility loop) save/restore around every speculative call;* ***TFields::isSub_*** *and* ***TFun::isSub_*** *may leave partial growth behind, but both are reached only through their restoring wrappers. The one non-restoring mutation on an error path is the* ***tenv.seenSub*** *record of step (2) — which is a write to a* local copy *of a frozen value, not to shared state (Lemma 9) — and the* ***seenExpansion*** *cache fills inside* ***collectTVars/expandToVectors****, which* are *rolled back by* ***restore*** *(the cache is part of* ***SubstState****) and are in any case semantics-preserving by Deep Dive C. This discharges, for the subtyping family, the corresponding slice of the main paper's Obligation "No Commitment on Failure".*

impl: `TTypeSubtyping.sk / file-head INVARIANT comment; all isSub wrappers`

3.2 The proved-pair memo: `subst.seenSub`

Definition 6 (Memo Discipline)**.** ***subst.seenSub*** *is a pair set inside* ***SubstState****. It is written at exactly one site — the wrapper's* STOk *path — and is captured and restored by* ***SavedSubst*** *together with* values/version*/the operation caches.*

impl: `TTypeSubst.sk / SubstState.seenSub; TTypeSubtyping.sk / TType::isSub`

Theorem 7 (Memo Soundness)**.** *Assume every non-memo acceptance is sound (the rest of this paper). Then a* ***subst.seenSub*** *hit is sound,* conditional on *Deep Dive E's monotone-reads theorem with its named residue MR-1.*

Proof. Two properties combine. *Timeline consistency*: because the set is part of the saved state, an entry present at query time was inserted on the current (never-rolled-back) timeline — entries inserted during exploration that failed were removed by the very `restore` that discarded the exploration (Lemma 5). So the entry witnesses an actual accepting run $\rho_i \to \rho_i^*$ with $[\![u_1]\!]_{\rho_i^*} \subseteq [\![u_2]\!]_{\rho_i^*}$, and every state evolution between ρ_i^* and the query state ρ_q was a sequence of legal transitions on the same timeline. *Transport*: Deep Dive E's monotone-reads theorem (registry row E-TRANSPORT) carries the validated inclusion across those transitions. Its residue applies verbatim here: MR-1 is precisely the case of a memo recorded while `freezeCandidates` was set — `Subst::get` then returns an *open candidate's* bound without sealing, so the recorded inclusion can quote a lower bound that later grows on the u_1 side. We do not re-derive it; the memo inherits MR-1 as a conditional hypothesis. Note the contrast with `tenv.seenSub`: no version key is needed here exactly because failed timelines are rolled back and successful growth is what the transport theorem is about. □

The freeze-window residue. The conditional hypothesis fails in the implementation: MR-1 is VIOLATED, witnessed end-to-end by an order-dependent unsound acceptance through a stale `seenSub` entry recorded in a freeze window (deep dive E; issue draft `issues/freeze-window-stale-memo-caches.md`). The lemma above therefore holds only *outside* freeze windows: `subst.seenSub` is unsound inside them until the memo is version-keyed or freeze-window writes are suppressed. The rest of Part I is unaffected — the proof isolates the failure to exactly this hypothesis.

3.3 Growth pre-passes are guidance

Step (5) of Definition 1 runs $\mathsf{growTVars}(u_1, u_2, \mathsf{contra})$ then $\mathsf{growTVars}(u_2, u_1, \mathsf{cov})$ before validating. By the main paper's Guidance-vs-Validation theorem, these passes require no soundness proof of their own: they only choose the state ρ' under which `Clauses::isSub` decides, and the failure path restores (Lemma 5). Three details of the audit are still worth recording, because each is a place where a reader might suspect a hidden acceptance. (i) The `isCovered` probes and the constraint-admissibility checks inside `growTVars` call `TType::isSub` speculatively; their verdicts influence only *which* disjuncts are grown, never the outer verdict, and each probe restores. (ii) The admissibility loop runs under `freezeCandidates`; by the main paper's never-widen lemma this makes it a query on already-determined candidates — but its *successful* probes may insert `subst.seenSub` memos while frozen, which is one of the two entry points of the inherited MR-1 hazard (Theorem 7). (iii) The `constraintErr` value threaded out of growth is used only to decorate the error message with hints on the failure path. (iv) When both sides are tuples of the same arity, the structural descent seeds each element variable from its positional pair only, not from the position-unioning method members (`pop`, `at`, ...) — a precision choice inside guidance: it changes which state validation decides under, never the verdict logic, so the lemmas of Part II are unaffected.

impl: `TTypeSubtyping.sk / growTVars, collectTVars, isSub_ (tuple positional seeding)`

4 The Cycle Cache: `tenv.seenSub` and Version Keying

The interesting cache is the second one. Its entry is recorded *at entry*, before the verdict is known — the implementation comment says it plainly: "if a cycle occurs, the subtyping relation is considered correct (by co-induction)." Soundness therefore needs two arguments: a *state* argument — a cache hit must not smuggle in a judgement recorded under different bindings — proved in this section, and a *logical* argument — why admitting a pair before it is proved is sound at all — which is §5.

4.1 The write audit: every binding write bumps the version

Lemma 8 (Version Audit)**.** *In `TTypeSubst.sk`, every assignment to `Subst.values` is immediately preceded by `version += 1`, and these are the only version bumps besides `gen/genForField/genForLiteralCommit` (which bump without writing `values`) and `restore` (which sets* version *and* values *jointly from a snapshot). Consequently, between two moments with no intervening `restore`,* version *unchanged implies* values *unchanged.*

Proof. By inspection of the four write sites: `grow` bumps at its single write (and its idempotent arm — $t_2 \vee t = t_2$ — returns *before* both the bump and the write, a fact §5 uses for cycle convergence); `get` bumps at both sealing writes (fresh-to-top and open-to-closed); no other method writes `values`. `TVar::setConstraint` is *not* a binding write: as established in Deep Dive E, it is a functional update producing a new `TVar` occurrence (and the new occurrence hashes differently, so cache keys

distinguish it — A-HASH). Callers outside the file (`TArrayLitTyping.sk`, `TVariadicTyping.sk`, `TypeCheckUtils.sk`) reach `values` only through `grow`/`get`, per the main paper's write-site audit. □

4.2 Why the naive reading fails, and what replaces it

The comforting reading — "the key $(\mathsf{version}, u_1, u_2)$ identifies the substitution state, so a hit replays a judgement made under the same bindings" — is *false*, as Deep Dive E established: `restore` rewinds the counter, so after `save` at v, a write to $v{+}1$, a restore to v, and a *different* write to $v{+}1$, two distinct binding maps have carried the same version number. Version numbers are reused across divergent exploration branches (and `gen` even reuses `TVar` identifiers across them). If `tenv.seenSub` were a global mutable table, entries recorded on a discarded branch would remain reachable on the surviving branch at colliding keys, and — worse — entries recorded at entry by checks that then *failed and restored* would be hit again at the identical rewound state, converting a rejection into a silent acceptance. Our first proof attempt followed the naive reading and foundered on exactly these two scenarios; we record that as a methodological finding: *the version key alone proves nothing.* What actually makes the cache sound is a pair of structural facts about how `tenv` and the saved substitutions travel, proved next.

4.3 Path scoping

Lemma 9 (Path Scoping)**.** *At any moment during a top-level `TType::isSub` run, the `tenv.seenSub` value visible to the currently executing `isSub_` invocation is exactly the ambient set of the top-level entry (empty in practice) plus one record* $(\mathsf{version}_A, a_1, a_2)$ *for each* strict ancestor *`isSub_` invocation A on the current call stack, recorded at A's entry. Records made by invocations that have already returned — successfully or not — are not visible.*

Proof. `TEnv` is a frozen class and `tenv` is passed by value through every signature in the file. The record statement `!tenv.seenSub = tenv.seenSub.set(...)` is Skip's functional update of the *local* binding `tenv` (contrast `subst.!seenSub = ...`, a true field mutation on the mutable `Subst`); the updated copy flows only into the callees of the recording invocation — `growTVars`, `expandToClauses`, `Clauses::isSub` and its strata — and evaporates when that invocation returns. A grep-complete audit shows `isSub_` is the only writer of the field in the codebase. Hence visibility is precisely "my ancestors' records": each ancestor recorded into its local copy before descending, and no other write exists. □

4.4 Record-point coherence

Lemma 10 (Record-Point Coherence)**.** *Suppose an `isSub_` invocation hits the cache:* $(\mathsf{version}_A, u_1, u_2) \in$ *`tenv.seenSub` with* $\mathsf{version} = \mathsf{version}_A$ *now. Then* values *now equals* values *at the moment ancestor A recorded the entry.*

Proof. Call A's record moment t_A, with state $(\mathsf{version}_A, X_A)$. Two sub-audits establish that every state on the current path between t_A and now with version $\mathsf{version}_A$ has $\mathsf{values} = X_A$.

(i) Save/restore locality. Every `SavedSubst` in the subtyping family is a local variable, created and consumed in the same invocation (Lemma 5); none is passed down or stored. Therefore every `restore` executed while A is pending targets a snapshot taken *after* t_A (the restoring invocation was entered after A, and its save postdates its entry). The wholesale snapshot/restore pairs outside the family (`saveAll`/`mcreateFromSaved` at the SKStore boundary, `clone` used by rendering

and refinement) never fire inside a pending `isSub_`: `clone` creates a detached substitution whose mutations do not touch the current one.

(ii) Version floor and determinism. By induction over the execution steps from t_A: every reachable state while A is pending has $\mathsf{version} \geq \mathsf{version}_A$, and if $\mathsf{version} = \mathsf{version}_A$ then $\mathsf{values} = X_A$. Writes only increase $\mathsf{version}$ (Lemma 8). A restore targets a post-t_A snapshot (v_s, X_s); by the induction hypothesis applied at snapshot time, $v_s \geq \mathsf{version}_A$, and if $v_s = \mathsf{version}_A$ then $X_s = X_A$; restoring re-establishes exactly that pair. `gen` bumps without writing, preserving both claims. So at the hit, $\mathsf{version} = \mathsf{version}_A$ forces $\mathsf{values} = X_A$. □

The two lemmas together repair the naive reading: cross-branch collisions cannot reach the cache (discarded branches' records were never visible, Lemma 9), failed checks leave no landmine (their records die with their stack frame), and a genuine hit is a *pending ancestor* re-encountered at *bit-identical bindings* (Lemma 10). What the version key buys is the second half: without it, a descendant could hit an ancestor's record after intervening growth, treating a judgement-in-progress under ρ as applicable under a strictly grown ρ' — the stale-cache regression class (#446) the key was introduced to kill. What remains to justify is the logic of accepting a pending pair at all.

impl: `TTypeSubtyping.sk / isSub_ (lines 1852, 1877); TEnv.sk / seenSub comment`

4.5 The head-depth refusal

Definition 11 (Head-Depth Refusal)**.** *Between the cycle-cache check and its record, `isSub_` counts, per ordered pair of* head constructor names *(both sides lone `TSInsts`), how many times the current path has entered that pair, in a map `tenv.subtypingHeadDepth` carried exactly like `tenv.seenSub` (a functional update of the local frozen-`TEnv` copy, so Lemma 9's path-scoping applies verbatim). Past the bound (`kMaxSubtypingHeadDepth` = 12) the check is* refused *with TS2589 rather than assumed related: two recursive aliases whose arguments grow at every level never repeat a `seenSub` entry, so without the bound their descent would be limited by nothing.*

impl: `TTypeSubtyping.sk / isSub_ (lines 1852-1877); kMaxSubtypingHeadDepth (line 18); TEnv.sk / subtypingHeadDepth`

For **L-SUB** the refusal is the sound direction — it can only reject — and it does not perturb the coinductive argument: a `tenv.seenSub` hit still names a pending ancestor at identical bindings (the record still precedes the descent), and the bound merely caps how many same-constructor entries a path accumulates before refusing. Its cost is completeness (deep finite nestings past the bound are rejected; the constant makes this rare), and its benefit is the termination remark of the honesty box below: the implementation carries its own divergence guard for the one recursion pattern the cycle cache cannot close.

5 Coinduction over the Value Semantics

5.1 What the semantics of a recursive type even is

The main paper's object semantics is a system of *equations*: $o \in [\![b]\!]$ is defined in terms of $\mathsf{get}(o, p) \in [\![\mathsf{get}(f)]\!]$ for field types that, for a recursive declaration (`type L = { next: L }` via `TSInst` unfolding), mention $[\![b]\!]$ again. The equations are monotone — every recursive occurrence of a denotation in the membership conditions is positive: field and index constraints use $[\![\cdot]\!]$ on the membership side, the call/new and brand clauses are atomic predicates on o, and nothing negates a membership — so by Knaster–Tarski both a least and a greatest solution exist, and the main paper's definition takes the greatest one (registry row SEM-GFP, which this paper consumes).

Definition 12 (SEM-GFP: Greatest-Fixpoint Reading (main-paper recap))**.** *The denotation* $[\![\cdot]\!]$ *is the* greatest *solution of the membership equations. This is the reading under which cyclic values inhabit recursive types:* `const x = {}; x.next = x` *builds a cyclic value that TypeScript and SKJS deliberately type as* `L`*, and the greatest solution contains exactly the values all of whose finite explorations are consistent with the type.*

Concretely we use the standard stratification. Define $[\![t]\!]^0 = \mathsf{Val}$ and let $[\![t]\!]^{n+1}$ be the main paper's membership conditions with every *structural* recursive membership — field values, index values, and (after §11) signature argument/result memberships — evaluated at level n, while tag tests, literal tests, callability, and brands are evaluated exactly. Unions, intersections, clause var-sets, and instance layers distribute through levels pointwise. Then $[\![t]\!] = \bigcap_n [\![t]\!]^n$ is the greatest solution (the defining operator preserves the intersection of this decreasing chain because each value's membership at level $n{+}1$ depends on finitely-branching conditions at level n; the union over object branches and clauses is over finite sets, so no non-ω-continuity arises on the relevant chain).

5.2 One-step soundness, and the reduction

Definition 13 (One-Step Soundness of a Stratum)**.** *Let* R *be a set of* TType *pairs ("the accepted pairs"). A stratum check is* one-step sound *if, whenever it accepts a pair* (x_1, x_2) *at state* ρ *with all of its recursive* `TType::isSub` *sub-checks resolving to pairs in* R*, then for every* n*:*

$$\big(\forall (a,b) \in R.\ [\![a]\!]_\rho \subseteq [\![b]\!]^n_\rho\big) \implies [\![x_1]\!]_\rho \subseteq [\![x_2]\!]^{n+1}_\rho.$$

Part II proves each stratum one-step sound (or names the failure). For readability Part II states the lemmas in the naive form "acceptance implies inclusion, given inclusion at the recursive positions"; each proof visibly uses the recursive hypotheses only inside one layer of membership conditions, which is exactly the shape Definition 13 requires, so the stratified reading is obtained by re-indexing, and we do not repeat the bookkeeping.

Theorem 14 (Coinductive Reduction)**.** *Assume A-HASH, SEM-GFP, the state discipline of §§3–4, and one-step soundness of every stratum reachable from* `Clauses::isSub`*. Then* ***L-SUB*** *holds: a top-level accepting run* $\rho \to \rho^*$ *satisfies* $[\![t_1]\!]_{\rho^*} \subseteq [\![t_2]\!]_{\rho^*}$*.*

Proof. Consider the finite call tree of the accepting run and let R be the set of pairs of all `TType::isSub` nodes that returned STOk, *each taken with the state at its verdict.* First flatten the state indexing: every node's verdict state precedes ρ^* on the same never-restored timeline (failed excursions restored, Lemma 5), so by Deep Dive E's monotone-reads transport (with its MR-1 proviso, and modulo the unsealed exact-value read of arm (a′), registry row **L-SUB-TSINST** — the transport premise that arm violates) it suffices to prove each node's inclusion at its own verdict state; we henceforth suppress ρ. Now show $\forall n.\ \forall (a,b) \in R.\ [\![a]\!] \subseteq [\![b]\!]^n$ by induction on n. Base: $[\![b]\!]^0 = \mathsf{Val}$. Step: fix $(a,b) \in R$ and $v \in [\![a]\!]$. The node for (a,b) accepted through one of the eight acceptance points. Fast path/top/TInter: by Lemma 4 the full inclusion $[\![a]\!] \subseteq [\![b]\!]$ holds outright, and $[\![b]\!] \subseteq [\![b]\!]^{n+1}$. The reflexive-`TSInst` arm (a′): the verdict-state inclusion needs unconstrained argument variables, and the flattening step above already carries its violated transport premise, so this case — hence the theorem — is conditional on registry row **L-SUB-TSINST**, exactly as the transport step is on MR-1. Memo (`subst.seenSub`): by Theorem 7 the entry witnesses an earlier accepting node, i.e. another element of R with the same pair; use its case. Cycle cache (`tenv.seenSub`): by Lemmas 9/10 the hit names a *pending ancestor* node with the same pair and bindings; that ancestor either eventually accepted — it did, since the whole run accepted and an ancestor of an executed node has its verdict on the accepting path — so the pair is in R, and

the induction is over n, not over the tree: circular use of the pair at level n is precisely what the induction hypothesis licenses. Delegated point (4): conditional on Obligation 2. Validation (5): `Clauses::isSub` accepted with its recursive sub-checks all STOk, i.e. with pairs in R; one-step soundness of the strata gives $v \in [\![b]\!]^{n+1}$ from the induction hypothesis. Finally $[\![a]\!] \subseteq \bigcap_n [\![b]\!]^n = [\![b]\!]$ by SEM-GFP, and instantiating at the root gives L-SUB. □

Honesty box. Theorem 14 is genuinely conditional on: MR-1 (inherited via the memo and via transport); O-ARRAYLIT; A-HASH; and one-step soundness of every stratum — Part II shows the last is where the substance and the violations live. Termination (that the cycle cache is *hit* rather than the growth diverging — convergence relies on the idempotent-grow early return noted in Lemma 8, and the argument-growing recursion the cache cannot close is refused by the head-depth bound, Definition 11) is a liveness property, out of scope for L-SUB.

Part II
The Strata

6 Primitives: Units and `TPrimValue::isSub`

Definition 15 (Primitive Checks)**.** *Both checks are invoked only on same-tag pairs (an* ***invariant*** *guards this; the caller guarantees it, §7). Units: accepted unconditionally.* ***TPrimValue::isSub***(p_1, p_2) *on literal sets:* $(-, \top)$ *accept;* $(\top, L_2)$ *reject;* (L_1, L_2) *accept iff every stored spelling in* L_1 *occurs in* L_2 *(string equality of normalized lexemes,* ***lits2.contains(elt.val)****). It touches no state.*
impl: TTypeSubtyping.sk / TPrimValue::isSub, STypes::isSub (unit arm)

Lemma 16 (Primitive Strata are Sound — Unconditionally)**.** *(i) For same-tag units* u_1, u_2*:* $[\![u_1]\!] = [\![u_2]\!]$*. (ii)* ***TPrimValue::isSub***$(p_1, p_2) = \mathsf{STOk} \implies [\![p_1]\!] \subseteq [\![p_2]\!]$.

Proof. (i) The three unit constructors have three distinct tags (undef, void, null), so same tag forces the same unit and singleton denotations coincide. (ii) $(-, \top)$: by the main paper's A0 every $v \in [\![p_1]\!]$ has p_1's tag, and $[\![\mathsf{prim}(k, \top)]\!]$ is the full tag set. (L_1, L_2): let $v \in [\![p_1]\!]$, so $v \approx (\mathsf{tag}(k), \delta_k(\ell))$ for some $\ell \in L_1$; the spelling ℓ occurs in L_2, hence $v \in [\![p_2]\!]$ by the same closure clause. This uses only that δ_k is a *function* of the spelling — **L-LIT**(i), the proved part — not injectivity: a duplicate value under two spellings can only cause spurious *rejection* (a completeness matter, L-LIT territory), never acceptance. No recursive positions exist, so one-step soundness is the plain statement. □

This stratum is the model case for the paper's staging: fully proved, no conditions, no state.

7 The Union Spine: `STypes::isSub`

Definition 17 (Tag-Aligned Covering)**.** ***STypes::isSub***(S_1, S_2) *runs* ***leq*** *over the tag-keyed maps: for each member* $s_1 \in S_1$*, if* S_2 *has no member at* $\mathsf{tag}(s_1)$*, fail (****STTooManySType****); otherwise dispatch the same-tag pair to the unit/****TPrimValue****/****TObject*** *stratum. Members of* S_2 *at tags absent from* S_1 *are ignored. Saves at entry, restores on failure.*
impl: TTypeSubtyping.sk / STypes::isSub; InternalKeyedArrayMap.sk / leq, iterDiffInterDiff

Lemma 18 (STypes Covering is Sound)**.** *If* ***STypes::isSub***$(S_1, S_2) = \mathsf{STOk}$ *and each accepted same-tag pair* (s_1, s_2) *satisfies* $[\![s_1]\!] \subseteq [\![s_2]\!]$ *(the sub-stratum obligation, one-step for objects), then* $[\![S_1]\!] \subseteq [\![S_2]\!]$.

Proof. Let $v \in [\![S_1]\!] = \bigcup_{s \in S_1} [\![s]\!]$, say $v \in [\![s_1]\!]$. By A0, $\mathsf{rtag}(v) = \mathsf{tag}(s_1)$. `leq` succeeded, so S_2 has a member s_2 at that tag — the keyed-map structure (at most one member per tag, iterated by sorted key) is what makes "the" member well-defined and the `invariant` in the code unreachable-else — and the sub-stratum accepted (s_1, s_2), giving $v \in [\![s_2]\!] \subseteq [\![S_2]\!]$. Ignoring S_2's extra tags is sound: they only enlarge the right side. One-step form: the object arm's hypothesis is used at level n, delivering $[\![s_2]\!]^{n+1} \ni v$, and unit/primitive arms are exact at every level. □

Completeness is not claimed and indeed fails (a literal member of S_1 covered by *no single* same-tag member cannot borrow from other tags — correct, by A1 there is nothing to borrow), but incompleteness is not our concern. Note the check inherits *no* condition from Deep Dive B's L-INTER-BOT refutation: no intersection is consumed on this path.

8 DNF Covering: `Clauses::isSub`

Definition 19 (Clause Covering). *`Clauses::isSub`(C_1, C_2): for every clause $\langle V_1 \mid S_1 \rangle \in C_1$, search C_2 for a clause $\langle V_2 \mid S_2 \rangle$ with (gate) $V_2 \subseteq V_1$ — the code's `inter1.vars.contains(inter2.vars)`, a superset test by the `ArraySet::contains` definition — and (body) one of: $S_1 = S_2 = \top$: match; $S_1 \neq \top, S_2 = \top$: match; $S_1 = \top, S_2 \neq \top$: skip this candidate; both $\neq \top$: match iff `STypes::isSub`(S_1, S_2) accepts. Accept iff every C_1-clause finds a match; restore on failure.*
impl: `TTypeSubtyping.sk / Clauses::isSub`

Lemma 20 (Clause Covering is Sound, Parametrically in TParam). *If `Clauses::isSub` accepts (with the* STypes *obligations of the matched pairs), then for* every *interpretation ι : TParam $\to \mathcal{P}(\mathsf{Val})$ of the type parameters, $[\![C_1]\!]^\iota \subseteq [\![C_2]\!]^\iota$ where $[\![\langle V \mid S \rangle]\!]^\iota = \bigcap_{p \in V} \iota(p) \cap [\![S]\!]$.*

Proof. Fix ι and $v \in [\![\langle V_1 \mid S_1 \rangle]\!]^\iota$ for some C_1-clause, and let $\langle V_2 \mid S_2 \rangle$ be its match. Gate: $V_2 \subseteq V_1$ gives $\bigcap_{V_1} \iota \subseteq \bigcap_{V_2} \iota$ — for *any* ι, which is the parametricity: the algorithm never inspects a parameter's meaning, so its verdict is uniform in ι, and instantiating parameters later (generic call sites, the `RenameClauses` step of §11) preserves the inclusion. Body: if $S_2 = \top$, $[\![S_2]\!] = \mathsf{Val}$ and we are done. Both concrete: Lemma 18 gives $[\![S_1]\!] \subseteq [\![S_2]\!]$ (one-step: at level $n{+}1$ via the hypotheses at n). The skipped case ($S_1 = \top$, S_2 concrete) rejects nothing by itself; it merely forces another candidate, and if none exists the check fails — sound. Union over C_1's clauses completes it. □

Two deliberate incompletenesses, noted for contrast with soundness: a C_1-clause must be covered by a *single* C_2-clause (no covering by unions of clauses — e.g. $T \mid$ `string` on the right never combines its parts), and the gate demands syntactic var-set containment rather than any semantic entailment between parameters. Both only reject.

9 Object Branches: `TObject::isSub`

Definition 21 (Branch Matching). *`TObject::isSub`(o_1, o_2): every branch $b_1 \in \mathsf{branches}(o_1)$ must satisfy `TFields::isSub`(b_1, b_2) for* some *$b_2 \in \mathsf{branches}(o_2)$; each failed pairing restores before the next candidate is tried, and overall failure restores to entry.*
impl: `TTypeSubtyping.sk / TObject::isSub`

Lemma 22 (Branch Matching is Sound). *If `TObject::isSub` accepts and each accepted pairing satisfies $[\![b_1]\!] \subseteq [\![b_2]\!]$ (one-step, the* TFields *obligation), then $[\![o_1]\!] \subseteq [\![o_2]\!]$.*

Proof. Membership in an object type is existential over branches ($o \in [\![\mathsf{object}(B)]\!]$ iff $o \in [\![b]\!]$ for some $b \in B$), so the $\forall\exists$ shape of the algorithm matches the semantics exactly: $v \in [\![b_1]\!]$ for some b_1, its matched b_2 gives $v \in [\![b_2]\!] \subseteq [\![o_2]\!]$. The per-pairing save/restore matters for a different reason than soundness of the verdict: a failed b_2-candidate may have committed growth (its field checks grow inference variables); restoring keeps the surviving pairing's state from being polluted by the failed one — without it the *accepted* state ρ^* would contain bindings from abandoned pairings, and while Guidance-vs-Validation says that could not falsify the verdict, it would degrade the solutions chosen. As with clauses, the single-branch covering is incomplete (no splitting a sub branch across super branches) but sound. □

10 Fields: `TFields::isSub`

This is the widest stratum. `isSub_` checks, in order: named string fields; the string index pair; the *effective* number index pair; the symbol index pair; named symbol fields (with the symbol index as their covering index); the call mode; the new mode. Named-field pairing uses `iterDiffInterDiff` over the name-keyed maps: $\mathsf{Left}(f_1)$ = sub-only field, $\mathsf{Right}(f_2)$ = super-only field, $\mathsf{Both}(f_1, f_2)$ = shared name. Throughout this section $o \in [\![b_1]\!]$ is fixed and we must establish each conjunct of $o \in [\![b_2]\!]$ per the main paper's object semantics; recursive `TType::isSub` calls are the one-step positions.

impl: `TTypeSubtyping.sk / TFields::isSub_, isSub_NamedFields`

10.1 Shared fields (Both)

The code: reject if f_1 optional and f_2 required; reject if $\neg f_1.\mathsf{access}.\mathsf{isCompatibleWith}(f_2.\mathsf{access})$; reject if $f_1.\mathsf{set} = \bot$ (readonly) and $f_2.\mathsf{set} \neq \bot$ (skipping the recursive check to avoid a TVar in $f_2.\mathsf{set}$ unifying its way to spurious success — a nice example of growth-avoidance *for precision*, not soundness); otherwise require $f_2.\mathsf{set} \leq f_1.\mathsf{set}$ (contravariant) and $f_1.\mathsf{get} \leq f_2.\mathsf{get}$ (covariant, against *bare* $f_2.\mathsf{get}$ even when both are optional — exact-optional semantics).

Lemma 23 (Shared-Field Soundness, Read Direction)**.** *The optionality gate, the access gate, and the covariant get check suffice for the* field clauses *of $o \in [\![b_2]\!]$ at the shared names, given the recursive inclusions.*

Proof. If f_2 is required, the gate makes f_1 required, so $\mathsf{has}(o, \mathsf{name})$ holds and (unless phantom) $\mathsf{get}(o, \mathsf{name}) \in [\![f_1.\mathsf{get}]\!] \subseteq [\![f_2.\mathsf{get}]\!]$ by the accepted recursive pair. If f_2 is optional, its clause constrains only the case $\mathsf{has}(o, \mathsf{name})$; when the key is present, membership in b_1 gives the value in $[\![f_1.\mathsf{get}]\!]$ *whether f_1 is optional or required* — this is exactly why checking against bare $f_2.\mathsf{get}$ (not $f_2.\mathsf{get} \cup$ `undefined`) is the right reading: the optional read's `| undefined` accounts for *absence*, and absence is vacuous here. The exactness comment in the code records the same argument. Phantom-access f_1: its get is $\top$, so the recursive check fails against any concrete $f_2.\mathsf{get}$ and the pair is rejected — restrict-only; the access gate independently rejects phantom-vs-declared (`FAPhantom` is compatible only with `FAPhantom`/`FANever`), and a phantom-vs-phantom acceptance constrains only presence on both sides. Private fields: compatibility demands the same owner name and file — nominal, strictly finer than the public case, so acceptance implies the same get/set checks ran; `FANever` as target is vacuously safe (nothing may access it; as a membership constraint it is an ordinary field and the get/set checks still ran).

impl: `TType.sk / FieldAccess::isCompatibleWith` □

Lemma 24 (Shared-Field Soundness, Write Direction — Conditional (registry row L-SUB-FIELD-S-SET))**.** *The contravariant set check ($f_2.\mathsf{set} \leq f_1.\mathsf{set}$) and the readonly gate are* not *consequences of membership: the main paper's semantics constrains values via get-types only, so Lemma 23 already delivers $o \in [\![b_2]\!]$ without them. What they buy is the mutation model: under **L-MUT**, a write through the b_2 view is checked against $f_2.\mathsf{set}$, and the contravariant condition guarantees any such accepted value is also legal for the underlying $f_1.\mathsf{set}$, i.e. writes through the super view cannot break invariants the sub view's clients rely on (the readonly gate is the $\bot$ instance: no value passes $\leq \bot$, so a readonly field never acquires writers by subsumption). Status: conditional on **L-MUT** — if the mutation model is ever made concrete, these checks are its proof obligations, not membership's.* □

10.2 Sub-only fields (Left) and super-only fields (Right)

Lemma 25 (Width and Optional-Absence Soundness)**.** *The* Left *and* Right *arms are sound for membership, given the recursive inclusions (read direction; their set-direction checks are conditional exactly as in Lemma 24).*

Proof. $\mathsf{Left}(f_1)$: a field only the sub declares. The super's membership conditions mention the name only through its *indexes*: if b_2 has an index covering the name (`indexForFieldName`: numeric-spelling names consult the number index first, then the string index — matching the covering semantics, under which a numeric-kind key must satisfy *both* when both exist; registry row `L-SUB-IDXNAME`, OPEN: the code checks against only the first-consulted index (`src/TType.sk:2135-2141`; call site `src/TTypeSubtyping.sk:1172`), so a numeric-named sub field is under-checked when the super has both a number and a string index. The routes to the under-check are narrow, three of them verified against the binary at the pin: a named-field-only source is rejected outright against an index-declaring super — the $(\mathsf{None}, \mathsf{Some})$ index-pair arm, which the implementation's own comment records as the guard (`src/TType.sk:2127-2134`); a declaration whose number and string indexes carry incompatible value types is rejected at the declaration itself; and reads of numeric-named keys resolve through the number index — the same index the width check consulted — while string-variable reads stay conservative. A validator-completeness gap, not a witnessed unsoundness; the "suffices" below therefore holds only under those guards, not in general), the code requires $f_1.\mathsf{get} \leq idx_2.\mathsf{get}$: then when $\mathsf{has}(o, \mathsf{name})$, the value lies in $[\![f_1.\mathsf{get}]\!] \subseteq [\![idx_2.\mathsf{get}]\!]$, which is what the index clause of $o \in [\![b_2]\!]$ demands of covered, non-named props. If b_2 has no covering index, the name is unconstrained by b_2 — the semantics is extensible — and accepting silently is exact. (The footnote in `TIndexesBase` about `"Infinity"`/`"NaN"` spellings classifying as string-kind is a key-*kind* discrepancy against TypeScript, argued unobservable there; for us it only changes *which* index the check runs against, and both candidate indexes' clauses are enforced by the index-pair checks, so soundness is insensitive to the classification.) $\mathsf{Right}(f_2)$ required: rejected — necessary, since extensible membership in b_1 says nothing about the name's presence. $\mathsf{Right}(f_2)$ optional: the code demands a b_1 index covering the name with $idx_1.\mathsf{get} \leq f_2.\mathsf{get}$. If $\mathsf{has}(o, \mathsf{name})$: the name is covered by idx_1 and not named in b_1 (this is the Right case), so membership in b_1 puts the value in $[\![idx_1.\mathsf{get}]\!] \subseteq [\![f_2.\mathsf{get}]\!]$; if absent, the optional clause is vacuous. Rejecting when b_1 has no covering index is again necessary: o may carry the key with an arbitrary value. For symbol fields the same two arms run with the symbol index as the covering index — same argument, one index kind. □

10.3 Index pairs and the covering semantics

Definition 26 (Index-Pair Check)**.** *isSub_IndexPair*(idx_1, idx_2)*:* $(-, \mathsf{None})$ *accept;* $(\mathsf{Some}, \mathsf{Some})$ *require get covariant and set contravariant;* $(\mathsf{None}, \mathsf{Some}(i_2))$ *probe* $\top \le i_2.\mathsf{get}$ *and* $i_2.\mathsf{set} \le \bot$ *— which fails for concrete index types and succeeds precisely when* i_2*'s get/set are inference variables (the* synthetic *index that object literals carry), unifying them with* $\top/\bot$*. The number-kind invocation uses* effective *indexes: a side lacking a number index falls back to its string index.*
impl: TTypeSubtyping.sk / isSub_IndexPair, isSub_ (number-index match); TType.sk / TField::createSyntheticIndex, kSyntheticIndexName

Lemma 27 (Index Covering Soundness)**.** *The three per-kind invocations establish the index clauses of* $o \in [\![b_2]\!]$*, given the recursive inclusions.*

Proof. Fix a prop $p \in \mathsf{props}(o)$ covered by an index of b_2 and not named by b_2's fields; we must place $\mathsf{get}(o, p)$ in that index's get-denotation. If p is named by b_1, the Left arm already did (Lemma 25). Otherwise p is anonymous on both sides and membership in b_1 constrains it only through b_1's indexes. Case symbol key: the symbol pair check gives $idx_1^{\mathrm{sym}}.\mathsf{get} \le idx_2^{\mathrm{sym}}.\mathsf{get}$, and $(\mathsf{None}, \mathsf{Some})$ with concrete i_2 is rejected — necessary, as o's symbol props are otherwise unconstrained; with synthetic i_2, growth makes $[\![i_2.\mathsf{get}]\!] = \mathsf{Val}$ under the accepted state, so the clause is vacuous and acceptance exact (the sealed-to-$\bot$ set side makes the literal's index reject all writers — restrict-only). Case string-kind key: only b_2's string index covers p; the string pair check plus the same $(\mathsf{None}, \mathsf{Some})$ analysis concludes. Case numeric-kind key: p must satisfy *every* covering index of b_2 — its number index and its string index. For b_2's string index: p is covered on the b_1 side by b_1's string index too (string covers both kinds), and the string pair check transfers it. For b_2's number index: o's constraint at p from b_1 is its number index if present, else its string index — exactly the *effective* number index — and the effective-pair check transfers it. The four-way match in `isSub_` enumerates precisely these fallbacks (both absent: nothing to transfer, and if b_2 has only a string index the string pair already handled numeric-kind keys). This is the point where the main paper's move to the covering semantics pays off: under the older per-kind-partition reading, the number-under-string fallback would have been an unsound borrowing; under covering it is the exact transfer. □

Corollary 28 (An Extensible Source Cannot Manufacture Closed Absence)**.** *Subtyping cannot admit an object source that leaves an uncovered key unconstrained into a* `Closed<T>` *target that proves the key absent. This corollary establishes registry row* ***L-SUB-CLOSED-ABSENCE*** *(conditional on the recursive* `isSub` *verdicts, as every field-stratum result is).*

Proof. `Closed<T>` represents every uncovered string or symbol key with a concrete index whose $\mathsf{get} = \bot$ (number keys fall back to the string index). Against a source with no corresponding index, the $(\mathsf{None}, \mathsf{Some}(i_2))$ arm must prove $\top \le i_2.\mathsf{get} = \bot$ and rejects. Against a source with a concrete index, the $(\mathsf{Some}, \mathsf{Some})$ arm requires $i_1.\mathsf{get} \le \bot$ and therefore accepts only when the source already proves the same absence. A source may also *name* an extra field the closed target proves absent: the width lemma's extra-field case checks a named source field with no target counterpart against the target's covering index — here the never-valued one — so it must establish $f_1.\mathsf{get} \le \bot$ and rejects any inhabited extra field, string- and symbol-named alike. Numeric-named extras consult the string never-index and nothing else: `closeObjectType` rejects every *declared* index (string, number, or symbol; SKJS 9033) before installing the never-indexes, and the synthetic number index is cleared — a valid closed target therefore has no number index at all, so the single-index consultation of L-SUB-IDXNAME cannot even arise here. Closing also drops a *synthetic* number index (`withNeverIndexes`) — an object-literal inference artifact carrying `TVar` get/set, describing

no declared capability — and this is absence-safe: with the number slot empty, numeric-kind keys fall under the string never-index for *admission* exactly as they do for the absence claim, so the claim and the gate move together. Hence an accepted admission never creates the absence fact on which the main paper's R-In true-branch pruning relies. Note the structural contrast with Finding F-CABSENT: indexes have no permissive middle state — an absent index reads $\top$ and $\top \leq \bot$ rejects — so the two-step conversion that defeats the callability brand has no analogue here. The pinned open-to-closed control is `tests/expect-error/closed-type.ts` / `testOpenIntoClosedRejected`.
impl: `TEnv.sk / closeObjectType; TType.sk / withNeverIndexes, createSyntheticIndex; TTypeSubtyping.sk / isSub_IndexPair, isSub_NamedFields (extra-field arm)` □

10.4 Callability: the lattice check and a violation

Definition 29 (Callability Check)**.** *`TFields::isSubCallability`*(m_1, m_2) *over the* Callability *lattice (§2), by arm (`src/TTypeSubtyping.sk:1255-1297`):* $(-, \mathsf{CUnknown})$ *accept;* $(\mathsf{CAbsent}, \mathsf{CAbsent})$ *accept;* $(\mathsf{CUnknown}, \mathsf{CAbsent})$ *accept* — permissive by design*, analyzed below;* $(\mathsf{CPresent}, \mathsf{CAbsent})$ *and* $(\mathsf{CFunction}, \mathsf{CAbsent})$ *reject;* $(\mathsf{CPresent}, \mathsf{CFunction})$ *and* $(\mathsf{CFunction}, \mathsf{CFunction})$ *accept;* $(\mathsf{CAbsent}, \mathsf{CFunction})$*,* $(\mathsf{CUnknown}, \mathsf{CFunction})$*,* $(\mathsf{CFunction}, \mathsf{CPresent})$*,* $(\mathsf{CAbsent}, \mathsf{CPresent})$*, and* $(\mathsf{CUnknown}, \mathsf{CPresent})$ *reject;* $(\mathsf{CPresent}(\sigma_1), \mathsf{CPresent}(\sigma_2))$ *recurses into `TFuns::isSub`. The same table serves `call` and `new` (invocations at `:1240-1251`).*
impl: `TTypeSubtyping.sk / TFields::isSubCallability`

Against the semantics of §2, every arm except one is exact or forced. The five rejections are semantically necessary (callable members cannot inhabit a $\neg$callable claim, and nothing short of a proof of [[Call]] may enter the positive CFunction brand — in particular $\mathsf{CUnknown} \not\leq \mathsf{CFunction}$, so *no unsupported value enters* CFunction; likewise a bare CFunction offers no signature to satisfy a CPresent target). $(-, \mathsf{CUnknown})$ is exact: the target claims nothing. $(\mathsf{CPresent}, \mathsf{CFunction})$ and the CFunction diagonal are exact: the target claims callability, which the source proves. The signature recursion is the subject of §11. The one permissive arm is not sound:

Finding 30 (F-CABSENT: Callables Can Enter a CAbsent Type)**.** $(\mathsf{CUnknown}, \mathsf{CAbsent})$ *is accepted: an unknown-callability source flows into a type whose mode claims* definitely not callable*. The arm exists for TS structural assignability — a plain object type flows into a class-instance type ({`a: number`} into `C`), and class instances carry* $\mathsf{call} = \mathsf{new} = \mathsf{CAbsent}$ *(`src/TEnv.sk:1003-1008`); the code comment names the cost explicitly (`src/TTypeSubtyping.sk:1264-1271`). Against §2's semantics it is a genuine non-inclusion: a callable $o \in [\![b_1]\!]$ (callables inhabit* CUnknown *branches — that is what makes the mode "unknown") is excluded from* $[\![b_2]\!]$ *by the* $\mathsf{call}(b_2) = \mathsf{CAbsent} \Rightarrow \neg\mathsf{callable}(o)$ *clause. And the violated clause is necessary in refinement: `narrowByTypeofFunction` drops* CAbsent *branches in the `typeof x === "function"` true case (`src/TType.sk:2438-2460`), so a callable accepted through `Function` $\to$ {} $\to$ `C` — two individually-accepted steps,* $(\mathsf{CPresent}, \mathsf{CUnknown})$ *then* $(\mathsf{CUnknown}, \mathsf{CAbsent})$ *— is narrowed to `never` in the branch the runtime takes. Witnessed end-to-end (registry row **F-CABSENT**; issue draft `issues/cabsent-brand-laundering.md`: SKJS accepts, Node takes the branch with a function value at type `never`, tsc rejects — the unsoundness is SKJS-specific).* CAbsent *is thus a* brand*: a claim refinement trusts as proof but subtyping enforces only against sources that themselves prove callability. The array brand has the same validation gap (Finding 31). Repair options: (a) reject* $(\mathsf{CUnknown}, \mathsf{CAbsent})$ *— sound, breaks TS structural assignability into class-instance types; (b) downgrade refinement's trust: treat* CAbsent *as droppable under `typeof "function"` only where every inhabitant is class-constructed (no structural admission route) — which is the same decision the array brand needs, so one resolution covers both findings. The same analysis applies verbatim to `new`/constructible.*

impl: TTypeSubtyping.sk / isSubCallability (CUnknown, CAbsent) arm; TType.sk / narrowByTypeofFunction; TEnv.sk / class-instance modes

10.5 The array brand: an omission

Finding 31 (F-BRAND: `isArray` Is Never Compared)**.** *`TFields::isSub_` contains no comparison of the `isArray` fields. The brand is stripped out of the named-field map at construction (`TFields::create` removes `__SKJS_isArray` and folds it into the flag), so it is not checked by the named-field arms either: a branch with* isArr = None *(or even* Some(false)*) is accepted against a branch with* isArr = Some(true) *whenever the structural checks pass. Against the main paper's brand clause (*isArr(b) = Some(x) ⇒ isArrayV(o) = x*) this is a non-inclusion; and as with F-CABSENT the violated clause is necessary elsewhere:* R-IsArray*'s* false *branch drops branches branded* Some(true)*, so a structural non-array impostor accepted at type `ReadonlyArray<T>` is typed `never` in the `else` of an `Array.isArray` test that its runtime takes. Exploitability is narrower than F-CABSENT — the impostor must structurally satisfy every array member, which TS programs rarely do accidentally, and tsc has no brand at all, so SKJS is here no* worse *than TS, merely less sound than its own model claims. (The construction-time lowering of an array-branded branch's modes to* CAbsent*, §2, rests on this same brand: the callability meet's ⊥ under the array brand is exactly as trustworthy as `isArray` itself.) Repair: compare brands in `isSub_` (*Some(x) ≤ Some(x)*; None ⋠ Some(x)*), or downgrade the brand clause to advisory and re-prove* R-IsArray *without the* Some(true)*-drop.*

impl: TTypeSubtyping.sk / TFields::isSub_ (no isArray arm); TType.sk / TFields::create (brand stripping)

Theorem 32 (Field Stratum, Assembled)**.** *`TFields::isSub` is one-step sound for the main paper's object membership* with the CAbsent clause restricted to callability-proving sources and the brand clause deleted*, in the read direction conditional only on **L-SUB-IDXNAME** at the numeric-named-field/dual-index corner (Lemmas 23, 25, 27 — the width lemma's named residual); its set-direction checks are the **L-MUT** obligations (Lemma 24); the two exceptions are exactly the F-CABSENT/F-BRAND violations, and the sound callability arms of §10.4 defer their signature recursion to §11.* □

11 Functions: `TFun::isSub` and a Behavioral Signature Semantics

The main paper's membership semantics treats signatures as opaque, so "`TFun::isSub` is sound" is not even a proposition against it: no set-inclusion consequence distinguishes right from wrong variance. This section supplies the missing semantics — the paper's main modeling contribution — proves the implementation's rules against it, and proposes it as a named extension of the main paper's object semantics.

Extension 33 (E-SIG: Call-Soundness Contract)**.** *Let* $\sigma = (\overline{\tau\ \text{tparams}},\ this^{?},\ \overline{p},\ \overline{q}^{\text{opt}},\ r^{?\,\text{rest}},\ ret)$*. Say an argument list* $\vec{a}$ conforms to σ *if* $|\vec{a}| \geq |\overline{p}|$ *and, for every position* i *that* σ covers *(the required* p_i*, then optional* q_j*, then — if a rest is present — every further position at the rest element type),* $a_i \in [\![\sigma_i]\!]$*; positions* beyond σ*'s coverage (possible only when* σ *has no rest) are unconstrained. Then define* $o \vDash \sigma$ *iff for every instantiation* θ *of* σ*'s type parameters satisfying their constraints, every this-value* $t \in [\![\theta\, this]\!]$ *(any* t *if this is absent), and every* $\vec{a}$ *conforming to* $\theta\sigma$*: if calling* o *with* $(t, \vec{a})$ *returns normally, the result is in* $[\![\theta\, ret]\!]$*. The proposed refinement of the main paper's object semantics: replace "signatures are opaque" by —* call(b) = CPresent(σ) *entails* callable(o) ∧ $o \vDash \sigma$*;* CFunction *entails* callable(o) *only (no signature claim — and `maybeGetCall` requires* CPresent*, so the checker never invokes through a bare* CFunction*);* CUnknown *entails nothing; dually for* new *with*

construction. (The CAbsent *clause is unchanged from §2 and its permissive subtyping arm is graded as Finding 30.) Two deliberate choices: extra trailing arguments are* unconstrained but permitted *— JS functions accept any arity, and the contract must quantify over them or arity-widening subsumption would be unsound; and only normal returns are constrained (exceptions untracked). In the stratified semantics of §5, argument memberships on the left of the implication are taken at full depth and the result membership at level n, keeping the operator monotone.*

Theorem 34 (Signature Variance is Sound for E-SIG)**.** *Suppose* `TFun::isSub`(f_1, f_2) *accepts, with its recursive pairs given (one-step). Then* $o \vDash f_1 \Rightarrow o \vDash f_2$, except *through the two arms isolated as Findings 35 and 36 below.*

Proof. Fix $o \vDash f_1$, an instantiation θ admissible for f_2, a this-value t, and $\vec{a}$ conforming to θf_2; we must land the result in $[\![\theta\, ret_2]\!]$. *Type parameters:* equal counts are enforced; `RenameClauses` identifies f_2's parameters with f_1's, so θ reads as an instantiation of the shared parameters; where both constraints exist, the contravariant check $cstr_2 \leq cstr_1$ (a recursive pair, transported by Lemma 20's parametricity through the renaming) makes θ admissible for f_1; where neither exists, trivial; the (Some, None) arm is Finding 36. *This:* the receiver is a four-state sum (`ThisParam`: `NoThis`, `Explicit`, `ImplicitClass`, `ImplicitShape`; `src/TType.sk:1384-1411`; arms at `src/TTypeSubtyping.sk:1992-2033`). A `NoThis` source accepts outright — its contract quantifies over every receiver. An `ImplicitShape` source against a non-shape target rejects (its receiver requirement is erased; nothing can be proven). The generic arm is contravariant, $this_2 \leq this_1$, with a *missing* target receiver read as $\top$ — a free-call permit, so the source's declared `this` must accept anything; a source demanding a real receiver is thereby rejected against a receiver-less target (TS accepts that; SKJS rejects). This arm is exactly E-SIG's quantification. The two remaining diagonals — `ImplicitShape`/`ImplicitShape` and `ImplicitClass`/`ImplicitClass`, both accepted without comparing receiver types — are justified only under an *attachment* reading of the receiver that E-SIG does not express: Finding 35. *Arguments:* we claim $\vec{a}$ conforms to θf_1. Both sides are first normalized by flattening tuple rests into positionals (`resolveSpreadToTuple`; conformance is invariant under this re-description of coverage, with the code's note that optionals preceding a flattened tuple become positionally required). $|\vec{a}| \geq |\overline{p}_2| \geq |\overline{p}_1|$, the latter enforced. For each position i covered by f_1: if f_2 covers i, the pairing loops checked $\sigma_{2,i} \leq \sigma_{1,i}$ contravariantly — required-vs-required/optional/rest, then optional-vs-optional/rest, then rest-vs-rest and the `STRecFunOptionalSpread` arm for f_1-optionals reached only by f_2's rest — so $a_i \in [\![\theta\sigma_{1,i}]\!]$; if f_2 does *not* cover i, then f_2 has no rest and i exceeds its coverage, but precisely this case is closed off by the two tail checks: f_1 may have neither more coverage ($|\overline{p}_1| + |\overline{q}_1| \leq |\overline{p}_2| + |\overline{q}_2|$) nor a rest when f_2 lacks one — the two "mysterious" arity rejections exist exactly to keep f_1-covered-but-f_2-unconstrained positions from arising. Positions beyond f_1's coverage are unconstrained by the contract. Hence the call is within o's contract for θf_1 and a normal result lies in $[\![\theta\, ret_1]\!] \subseteq [\![\theta\, ret_2]\!]$ by the covariant return pair. The `cgNodes` edge recording on success is a results-accumulator write, not a `SubstState` write — it does not perturb Part I's audits. □

Finding 35 (F-THIS-IMPLICIT: The Attach-Only Receiver Arms)**.** *The* `ImplicitShape/ImplicitShape` *and* `ImplicitClass/ImplicitClass` *diagonals accept without comparing receiver types. Under E-SIG as stated they are unjustified: the target's contract quantifies over receivers in* $[\![this_2]\!]$*, and nothing relates* $this_1$ *to it (for* `ImplicitClass` *the receivers may be outright incompatible — the arm exists so that generic-container variance and subclass overrides survive, since a contravariant receiver check would reject* `ReadonlyArray<Dog>` $\leq$ `ReadonlyArray<Animal>`*). What justifies them is a different receiver model: an* attachment *semantics in which a method's receiver at a call is the very object it was accessed on, so a method carried from a* b_1*-view of an object to a* b_2*-view*

of the same *object still runs on that object. The checker enforces the model's premise separately — shape methods are attach-only by construction (their carrier type is erased and a detached use rejects), and call sites check the receiver of* `Explicit/ImplicitClass` *functions — but E-SIG has no clause expressing "receiver = carrier", so the two diagonals cannot be graded against it. Named residual: extend E-SIG with an attachment clause and prove the diagonals against it, or restrict them. Status recommendation:* ASSUMED *(the attachment premise), pinned tests recommended.*
impl: `TTypeSubtyping.sk / TFun::isSub_ (thisParam match); TType.sk / ThisParam`

Finding 36 (F-TPARAM: Constraint Dropping)**.** $(\mathsf{Some}(cstr_1), \mathsf{None})$ *on a type-parameter pair is accepted:* `<T extends string>(x: T) => T` *passes as* `<T>(x: T) => T`*. An f_2-caller may instantiate outside $cstr_1$, exercising the sub's body off its contract; E-SIG does not justify the arm. (The subsequent positional checks run with the shared parameter* rigid*, so they cannot recover the gap.) Same severity shape as F-THIS. Status recommendation: named residual, pinned test recommended.*
impl: `TTypeSubtyping.sk / TFun::isSub_ (tparam cstr match)`

With E-SIG ratified, the callability arms of §10.4 other than F-CABSENT's are also fully justified: $(\mathsf{CPresent}(\sigma_1), \mathsf{CPresent}(\sigma_2))$ reduces to $\sigma_1 \leq \sigma_2$ under Theorem 34; $(\mathsf{CPresent}, \mathsf{CFunction})$ and the CFunction diagonal are exact, since the target claims callability only and the source proves it (E-SIG assigns CFunction no signature clause); and the rejections are forced — CUnknown does not entail callable, so nothing unproven enters the positive brand, and a bare CFunction offers no signature to satisfy a CPresent target.

12 Variance Plumbing: Why Each Side Expands the Way It Does

Step (5) expands u_1 with `Covariant()` and u_2 with `Contravariant()`; `expandToVectors` resolves each `TVar` through `Subst::get` at that variance (covariant: seal-and-return the binding, $\top$ if fresh; contravariant: return the binding unsealed, $\bot$ if fresh), then applies the variable's constraints via `TType::inter`.

Lemma 37 (Directional Reads)**.** *Validation is sound iff the sub-side expansion* over-*approximates and the super-side expansion* under-*approximates: from $[\![u_1]\!] \subseteq [\![E_{\mathrm{cov}}(u_1)]\!]$, $[\![E_{\mathrm{contra}}(u_2)]\!] \subseteq [\![u_2]\!]$, and $[\![E_{\mathrm{cov}}(u_1)]\!] \subseteq [\![E_{\mathrm{contra}}(u_2)]\!]$ (the validated inclusion) the target follows by composition; with either direction flipped, the composition breaks — e.g. reading a sub-side open variable contravariantly would validate $\alpha \leq$* `string` *from a still-growing bound* `"a"`*, later falsified when α grows to* `"a"` $\mid 1$*; reading a fresh super-side variable covariantly would seal it to $\top$ and accept everything against a variable whose solution is not yet chosen. The implementation's assignment is the correct one.* □

The two approximation facts themselves decompose as follows. *Covariant/sub side:* Deep Dive C's T-EXP proves expansion over-approximates for modes `None`/`Covariant`, conditional on (among others) **L-INTER** and **L-INTER-BOT** — and Deep Dive B *refuted* the $\bot$-clause **L-INTER-BOT** in its live strata (incomparable signatures return $\bot$; the beyond-precision num-spelling stratum). The exposure here is concrete: `get`'s constraint application $\mathsf{inter}(binding, cstr)$ can under-shoot the true intersection through those strata, shrinking the covariant expansion below $[\![\alpha]\!]$ and validating vacuously. We name the residue:

Obligation 38 (O-GETCOV: Covariant Reads Over-Approximate)**.** $[\![\alpha]\!]_\rho \subseteq [\![$`Subst::get`$(\alpha, \mathsf{cov})]\!]_{\rho'}$. *Follows from the instance-semantics axiom plus L-INTER/L-INTER-BOT; open on the remaining live* ***L-INTER-BOT*** *strata (incomparable signatures; beyond-precision num-spelling), which the constraint application can still hit. Status:* OPEN*, inherited.*

Obligation 39 (O-GETCONTRA: Contravariant Reads Under-Approximate)**.** $[\![\texttt{Subst::get}(\alpha, \mathsf{contra})]\!]_\rho \subseteq [\![\alpha]\!]_\rho$*, reading* $[\![\alpha]\!]$ *as the eventual committed solution:* $\perp$ *for fresh variables is trivially below anything; an* SVOpen *bound is below the final value because bounds only grow (Deep Dive E's transitions)* — but *the constraint intersection applied on the way out must not* enlarge *past* $[\![\alpha]\!]$*. This is a meet lower-bound property, not the over-approximation supplied by L-INTER/L-INTER-BOT, and the semantics of "eventual solution" for variables that are never sealed needs the main paper to fix a convention. Deep Dive C explicitly assigned this read to L-SUB; we discharge its use in Lemma 37 and leave the two flagged sub-claims open. Status:* OPEN.
impl: `TTypeSubst.sk / Subst::get; TType.sk / expandToVectors`

13 Status and Recommended Ledger Edits

13.1 Status by stratum

Stratum / item	Result here
Dispatch fast paths	Lemma 4
Restore-on-failure	Lemma 5 (family-wide audit)
`subst.seenSub`	Theorem 7 (§3.2)
`tenv.seenSub`	Lemmas 8, 9, 10: hit = pending ancestor at identical bindings
Coinduction	Theorem 14: L-SUB reduces to one-step strata
Head-depth refusal	Definition 11: rejection-only divergence guard, path-scoped
Growth pre-passes	guidance only (main paper theorem); MR-1 entry point noted; tuple positional seeding
Primitives, units	Lemma 16
STypes covering	Lemma 18
Clauses DNF + var gate	Lemma 20, parametric in TParam
`TObject` branches	Lemma 22
Fields, read direction	Lemmas 23, 25, 27 (incl. covering indexes, synthetic-index probes, access/phantom/private)
Fields, write direction	Lemma 24
Callability lattice	Definition 29: all arms exact or forced except one
(CUnknown, CAbsent) arm	Finding 30 + executed witness
Array brand	Finding 31 + composite witness
`TFun` variance	Theorem 34 against E-SIG
`this`/tparam-cstr arms	Findings 35, 36
Variance plumbing	Lemma 37; Obligations 38, 39
Array-literal decision	Obligation 2

13.2 Stratum outcomes and recommendations

1. *The L-SUB surface decomposes as*: L-SUB-ARCH (caches, dispatch, coinduction; PROVED here mod A-HASH — except the `subst.seenSub` component, VIOLATED via the witnessed MR-1 until the memo is version-keyed); L-SUB-PRIM (PROVED); L-SUB-DNF (STypes/Clauses/`TObject` skeletons; PROVED modulo lower strata); L-SUB-FIELDS-GET (PROVED); L-SUB-FIELDS-SET (conditional on L-MUT); L-SUB-MODES (VIOLATED, F-CABSENT); L-SUB-BRAND (VIOLATED, F-BRAND); L-SUB-FUN (PROVED against E-SIG, residuals F-THIS-IMPLICIT/F-TPARAM); L-SUB-EXPVAR (OPEN, O-GETCOV/O-GETCONTRA); L-SUB-ARRAYLIT (OPEN).

2. *Recommended model extensions*: E-SIG (adopt Extension 33, replacing "signatures are opaque"); A-HASH (hash-consing identity) — both are registry entries. Note that adopting E-SIG strengthens the main paper's monotonicity axiom: its deflection of the contravariance counterexample ("membership never inspects parameter types") stops being automatic and must cite the stratified-level bookkeeping of §5.

3. *Define when refinement may rely on brands*: the CAbsent and array brands are claims refinement trusts as proofs but subtyping enforces only against sources that themselves prove the property; either reject the permissive admissions ((CUnknown, CAbsent); the missing brand arm) or downgrade the R-TypeOf/R-IsArray drops to brand-insensitive narrowing — one decision covers both findings; wire the executed F-CABSENT witness and F-BRAND's composite sketch as pinned tests either way.

4. *Cross-reference*: the L-INTER-BOT row lists O-GETCOV as a downstream dependent, so that repairing the remaining L-INTER-BOT strata is understood to be on L-SUB's critical path, not only refinement's.

Registry. The authoritative, current statuses are maintained in the registry (`docs/ledger.yaml`), rendered as the main paper's Assumptions and Proof Obligations tables; this paper is cited there as evidence. The registry rows it supports:

Row	Statement	Status
L-SUB	`Clauses::isSub` acceptance implies semantic inclusion under the current state.	OPEN
E-SIG	Behavioral call-soundness semantics for function signatures (required for function subtyping to be a soundness statement).	ASSUMED
A-HASH	`TType::compare` equality is trustworthy (XXHash-only fast paths).	ASSUMED
L-SUB-FIELDS-SET	`isSub`'s field arms preserve write capability downward: acceptance implies each target field's set-type is below the source's, and readonly fields acquire no writers.	OPEN
L-SUB-CLOSED-ABSENCE	`isSub` admission cannot manufacture a `Closed<T>` absence fact: acceptance requires the source to prove $\mathsf{get} = \bot$ at every key the target proves absent.	OPEN
O-GETCOV	Soundness of the covariant variable read in subtyping (inherits L-INTER-BOT's violated $\bot$-clause).	OPEN
O-GETCONTRA	Soundness of the contravariant variable read in subtyping.	OPEN
O-ARRAYLIT	Soundness of the array-literal decision procedure inside subtyping.	OPEN
F-CABSENT	The CAbsent brand-like claim accurately excludes callables: refinement drops CAbsent branches under `typeof "function"`, but subtyping's permissive (CUnknown, CAbsent) arm admits callables into branded types.	VIOLATED
F-BRAND	Subtyping never compares the array brand, though refinement narrows on it.	VIOLATED
F-THIS-IMPLICIT	Subtyping between two shape receivers, or two class receivers, with implicit `this` parameters accepts without comparing their receiver types.	ASSUMED
L-SUB-IDXNAME	A numeric-named field is checked against both a number and a string index when the supertype has both (soundness of the width lemma's numeric-named case).	OPEN
A-INTERSEM	$[\![\mathsf{TInter}(\bar{t})]\!] \subseteq \bigcap [\![t]\!]$ — the instance-layer upper bound the dispatcher's fast paths assume.	ASSUMED
L-SUB-TSINST	The reflexive `TSInst` fast path in `isSub` (same name, arguments equal after `peekRaw`) is sound.	VIOLATED

Imports (review contract). The named statements this paper assumes from elsewhere. A reviewer treats each as an axiom, checking citations against the registry's canonical statement; no other paper's internals are in scope:

Row	Canonical statement	Owner	Status
L-INTER	When `inter` returns a non-$\bot$ result, that result contains the semantic intersection of its operands, provided every recursive call that returned $\bot$ had disjoint operands (L-INTER-BOT).	B	OPEN
L-INTER-BOT	A $\bot$ result from `inter` implies semantic disjointness.	B	VIOLATED
L-EXP	Expansion preserves or enlarges semantics (modes `None`/Covariant); residual instances keep theirs.	C	PROVED
A-CTOR-CALL	Class-constructor values carry CPresent construct and CFunction call modes, consistent with [[Call]]/[[Construct]].	A	PROVED
L-LIT	Literal spellings: (i) δ is a function; (ii) injectivity modulo $\approx$ after normalization; (iii) falsiness-exactness of the lexeme test.	G	VIOLATED
L-MUT	Field writes are governed by set-types; get-only membership is sound for snapshot reads.	main	ASSUMED
OBL-MONOTONE	Monotone Reads Under Growth: judgements validated under ρ survive later legal transitions.	E	VIOLATED
OBL-RESTORE	No Commitment on Failure: failing and exploratory paths restore all relevant state.	E	PROVED
MR-1	No `subst.seenSub` memo recorded under `freezeCandidates` may outlive growth of an open candidate readable from its left side.	E	VIOLATED
E-ARR	Delayed array-literal forms keep their semantics under expansion.	C	ASSUMED
SEM-GFP	The set-theoretic semantics is the greatest solution of the recursive membership equations.	main	PROVED
E-TRANSPORT	Monotone-reads transport: an inclusion validated under ρ survives later legal transitions on the same timeline, except for the cache behavior recorded by MR-1.	E	PROVED

13.3 Failed-attempt log

Recorded per series policy: (i) the global "version-as-state-hash" argument for `tenv.seenSub` fails (two concrete counterexample schemas, §4.2); the successful replacement needs the frozen `TEnv` threading and save/restore locality, neither of which is about the version at all. (ii) A membership-only proof of the field stratum fails at exactly two clauses (the CAbsent clause under its permissive admission arm, and the brand) — both times because the main paper priced those clauses into *refinement* proofs; the failures are reported as violations rather than weakened into assumptions. (iii) A proof of `TFun::isSub` against the opaque-signature semantics is not merely hard but type-incorrect — the conclusion is not expressible; hence E-SIG. (iv) An unconditional proof of the covariant read (O-GETCOV) fails against Deep Dive B's L-INTER-BOT counterexamples; we did not find a validator-local workaround, since the constraint application lives inside `Subst::get`.